



Rapport de Projet

Operational Research S2

FRELAT Jade

Master 1 Informatique

10 mai 2026

Table des matières

1	Introduction	2
2	Problème du flot maximum et de la coupe minimum	3
2.1	Algorithme de Ford-Fulkerson	3
2.2	Exercice : flot maximum	4
3	Problème du flot maximum avec coût minimum	5
3.1	Algorithme Repeated Augmenting Paths avec Bellman-Ford	5
3.2	Algorithme Repeated Augmenting Paths avec Dijkstra	6
3.3	Exercice : problème d'affectation	6
3.4	Exercice : flot à coût minimum	7

1. Introduction

Étant donné un graphe orienté où chaque arête possède une capacité maximale et éventuellement un coût, l'objectif est de faire circuler une quantité maximale de flot entre une source s et un puits t .

Dans ce projet, les graphes sont représentés par un ensemble d'arêtes. Chaque arête est décrite par un quadruplet :

$$(u, v, c, w)$$

où :

- u est le sommet de départ,
- v est le sommet d'arrivée,
- c est la capacité de l'arête,
- w est le coût de l'arête.

2. Problème du flot maximum et de la coupe minimum

2.1 Algorithme de Ford-Fulkerson

L'algorithme de Ford-Fulkerson permet de résoudre le problème du flot maximum et de la coupe minimum.

L'idée est de rechercher successivement des chemins augmentants P . Un chemin augmentant est un chemin reliant la source s au puits t dans le graphe résiduel, dont toutes les arêtes ont une capacité strictement positive.

Dans ce projet, ces chemins sont recherchés à l'aide d'un parcours en largeur (BFS).

La quantité de flot pouvant être ajoutée sur ce chemin est :

$$\delta = \min_{e \in P} \text{cap}(e)$$

où $\text{cap}(e)$ désigne la capacité de l'arête e dans le graphe résiduel du flot f .

Le flot est ensuite augmenté de δ sur ce chemin, puis le graphe résiduel est mis à jour. Le processus est répété jusqu'à ce qu'il n'existe plus de chemin augmentant.

Le théorème max-flow min-cut dit que la valeur du flot maximum correspond à la valeur de la coupe minimum et que l'ensemble des sommets atteignables depuis s dans le graphe résiduel du flot final définit une coupe C .

Dans ce projet, ces sommets sont trouvés grâce à un BFS. La coupe minimum est représentée par l'ensemble des arêtes reliant C à $G \setminus C$.

Complexité

$$O(nm^2)$$

2.2 Exercice : flot maximum

On considère le graphe E suivant :

```
start_nodes = [0, 0, 0, 1, 1, 2, 2, 3, 3]
```

```
end_nodes   = [1, 2, 3, 2, 4, 3, 4, 2, 4]
```

```
capacities  = [20, 30, 10, 40, 30, 10, 20, 5, 20]
```

L'objectif est de calculer un flot maximal entre les sommets 0 et 4.

Pour cela, il suffit d'exécuter :

```
max_flow_min_cut_ford_fulkerson(E,s,t)
```

On obtient un flot maximal de :

3. Problème du flot maximum avec coût minimum

L'algorithme *Repeated Augmenting Paths* permet de résoudre le problème du flot maximum avec coût minimum.

L'idée est similaire à celle de l'algorithme de Ford-Fulkerson, à la différence que l'on n'augmente plus le flot sur n'importe quel chemin augmentant, mais sur le chemin augmentant de coût minimum.

Or, dans le graphe résiduel, les arêtes inverses possèdent des coûts négatifs. Il est donc nécessaire d'utiliser un algorithme capable de gérer des poids négatifs.

3.1 Algorithme Repeated Augmenting Paths avec Bellman-Ford

Une première idée consiste à rechercher les chemins augmentants à l'aide de l'algorithme de Bellman-Ford.

Complexité

— Bellman-Ford :

$$O(nm)$$

— Min-cost max-flow :

$$O(Fnm)$$

où F est la valeur du flot maximal.

3.2 Algorithme Repeated Augmenting Paths avec Dijkstra

Bellman-Ford étant relativement coûteux, une seconde approche consiste à utiliser Dijkstra.

Le problème est que Dijkstra ne fonctionne pas avec des coûts négatifs. On renormalise donc les coûts grâce à des potentiels $h(v)$.

Les nouveaux coûts sont définis par :

$$c'(u, v) = c(u, v) + h(u) - h(v)$$

où :

- $c'(u, v)$ est le coût renormalisé de l'arête (u, v) ;
- $c(u, v)$ est le coût initial ;
- $h(x)$ est la distance entre s et x calculée par Bellman-Ford.

Ces nouveaux coûts deviennent alors positifs ou nuls, ce qui permet d'utiliser Dijkstra.

Complexité

- Bellman-Ford :

$$O(nm)$$

- Dijkstra :

$$O(m \log n)$$

- Min-cost max-flow :

$$O(nm + Fm \log n)$$

3.3 Exercice : problème d'affectation

On considère le problème d'affectation suivant :

- 10 personnes ;
- 8 tâches ;

- un coût d'affectation pour chaque couple personne/tâche.

La matrice des coûts est :

```
cost = [
[90, 76, 75, 70, 50, 74, 12, 68],
[35, 85, 55, 65, 48, 101, 70, 83],
[125, 95, 90, 105, 59, 120, 36, 73],
[45, 110, 95, 115, 104, 83, 37, 71],
[60, 105, 80, 75, 59, 62, 93, 88],
[45, 65, 110, 95, 47, 31, 81, 34],
[38, 51, 107, 41, 69, 99, 115, 48],
[47, 85, 57, 71, 92, 77, 109, 36],
[39, 63, 97, 49, 118, 56, 92, 61],
[47, 101, 71, 60, 88, 109, 52, 90]
]
```

Le graphe de flot est construit de la manière suivante :

- une source reliée à chaque personne avec une capacité 1 et un coût nul ;
- une arête entre chaque personne i et chaque tâche j de capacité 1 et de coût `cost[i][j]` ;
- une arête entre chaque tâche et le puits de coût nul et de capacité dépendant du contexte.

1. Sans capacité sur les tâches : On obtient un coût minimal total de :

370

2. Avec une capacité de 2 pour chaque tâche : On obtient un coût minimal total de :

404

3. Avec une capacité d'au moins 1 pour chaque tâche :

338

3.4 Exercice : flot à coût minimum

On considère le graphe :

```
start_nodes = [0, 0, 1, 1, 1, 2, 2, 3, 4]
```



```
end_nodes    = [1, 2, 2, 3, 4, 3, 4, 4, 2]
capacities   = [15, 8, 20, 4, 10, 15, 4, 20, 5]
unit_costs   = [4, 4, 2, 2, 6, 1, 3, 2, 3]
```

On ajoute :

- une source s reliée au sommet 0 avec une capacité 20 ;
- un puits t avec :

$(3, t)$ de capacité 5

$(4, t)$ de capacité 15

- une arête (t, s) .

Les objectifs sont :

1. calculer le min-cost max-flow entre (t, s) : max flot =20, min cost=150
2. calculer le min-cost max-flow entre $(0, 4)$: max flot =23, min cost=187
3. déterminer l'arête la plus vitale, c'est-à-dire celle dont la suppression réduit le plus la valeur du flot maximal entre s et t . : (0-1)